



Toronto Metropolitan University

Faculty of Engineering & Architectural Science

Department of Electrical, Computer and Biomedical Engineering

Course Title:	Computer Network I
Course Number:	COE768 - 152
Semester/Year (e.g. F2016)	F2025

Instructor:	Dr. Truman Yang
Teaching Assistant (TA)	Safwan Hasan - safwan.hasan@torontomu.ca

<i>Assignment/Lab Number:</i>	Final Project
<i>Assignment/Lab Title:</i>	P2P Application

<i>Submission Date:</i>	November 28th, 2025
<i>Due Date:</i>	November 28th, 2025

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Maisam Abbas	Syed	501103255	04	

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <http://www.ryerson.ca/senate/current/pol60.pdf>

TABLE OF CONTENTS

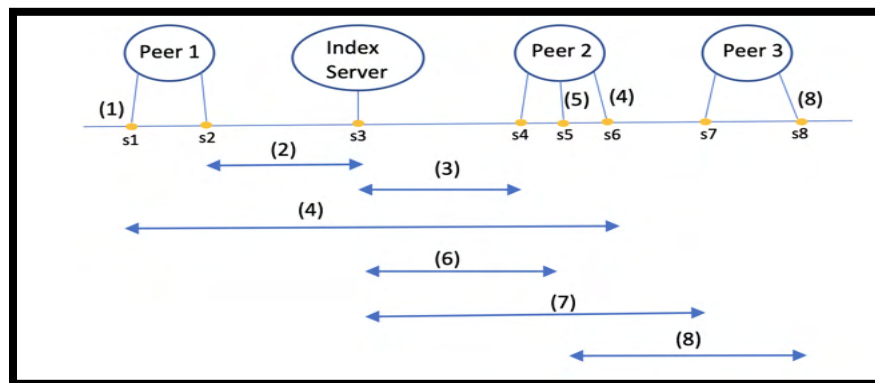
Introduction.....	3
Theory.....	4
Programs.....	7
Observation & Analysis.....	11
Conclusions.....	18
Appendix.....	20
peer.c.....	20
index.c.....	28

INTRODUCTION

The objective of this project is to create a peer-to-peer (P2P) program that allows a network of computers to exchange material. An index server and several peers are the two primary parts of the system. By maintaining a registry of available content and the addresses of peers (referred to as content servers) that have it, the Index Server acts as a central directory. Peers can operate as both content servers (making content accessible to others) and content clients (downloading content).

This P2P application's peer component needs to have advanced socket handling since it has to handle several network operations at once, including serving as a server (listening for connection requests from other peers) and acting as a client (querying the Index Server and requesting downloads). Additionally, it needs to continuously handle user input from the command line (stdin). The project makes use of two crucial system calls, *getsockname* and *select*, to effectively handle these various I/O activities without allocating a thread to each.

Figure 1.1: Server (Index) connections to multiple ports (Peer)



In order to enable inter-process communication via the network, this P2P system's implementation depends on basic ideas of computer networks and socket programming. A hybrid communication protocol method is employed by the application:

- **Index-Server Communication:** The User Datagram Protocol (UDP) is used to facilitate communication between a peer and the Index Server (for registration and content look-up). UDP is a connectionless protocol that facilitates quick, low-overhead communication for straightforward request-reply exchanges.
- **Content Download:** The Transmission Control Protocol (TCP) is the foundation for the actual transfer of content between a content client and a content server. For correct file transfer, TCP, a connection-oriented protocol, guarantees dependable, in-order delivery.

Application programs can access these network services through the Socket API. In particular, the project calls for the usage of stream (TCP) sockets for content download and UDP sockets for Index Server communication, together with the three-way handshake for connection formation and related methods like `socket()`, `bind()`, `listen()`, `connect()`, `read()`, and `write()`.

THEORY

A unique Application Layer Protocol is designed in order to facilitate the particular interactions (registration, search, and content download) needed by the P2P application. Every communication message is contained in a standard Protocol Data Unit (PDU) format, regardless of whether it uses TCP (for content download) or UDP (for Index Server communication). A consistent message structure across the various underlying transport protocols is ensured by this stacking.

I. Protocol Data Unit (PDU) Structure

There are two main fields in the basic PDU format.

- **Type Field:** A one-byte field that specifies the message's particular purpose or function. To handle the different application functions (such as registration requests, search replies, content transfers, etc.), the project makes use of eight different PDU kinds.
- **Data Field:** The real application-specific data is stored in a variable-length field.

Figure 2.1: A protocol data unit (PDU) has the illustrated simple format



Table 2.2: Summary of the eight (8) PDU types and their function/implementation

PDU type	Function	Direction
R	Content Registration	Peer to Index Server
D	Content Download Request	Content Client to Content Server
S	Search for content and the associated content server	Between Peer and Index Server
T	Content De-Registration	Peer to Index Server
C	Content Data	Content Server to Content Client
O	List of On-Line Registered Content	Between Peer and Index Server
A	Acknowledgement	Index Server to Peer
E	Error	Between Peers or between Peer and Index Server

II. Data Field Specification - (Registration)

Depending on the PDU type, the Data field's size changes:

- **PDU's for Control and Signalling (all types except 'C'-type):** The maximum data field size for messages used for registration, lookup, and acknowledgement is 100 bytes. For sending brief control information like file names, IP addresses, and port numbers, this is adequate.

- Content Data PDU ('C'-type): During a download, this PDU is specifically made to transport the actual content data (file chunks). The size of the material being transferred determines the size of its data field, which might be much bigger than the underlying TCP maximum segment size (MSS).

The content file needs to be divided into smaller sections in order for the TCP connection to reliably manage massive file transfers. Thus, until the complete file is transferred, the C-type PDU will transport these portions over several packets. This method of file segmentation and transmission is in line with common network programming techniques, including those shown in earlier socket programming labs (like Lab 3).

Figure 2.3: PDU - Content Registration

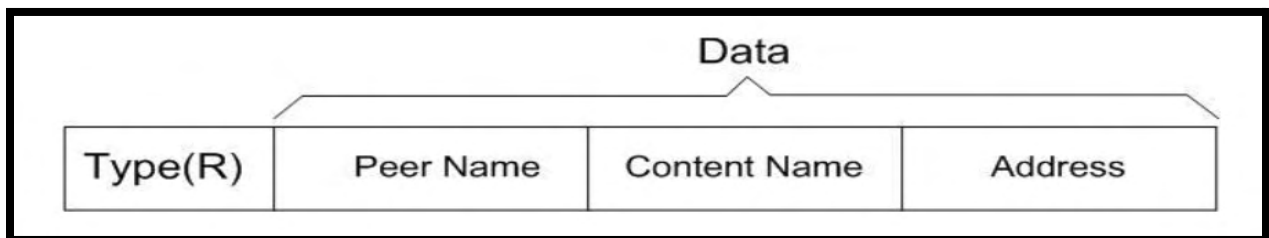
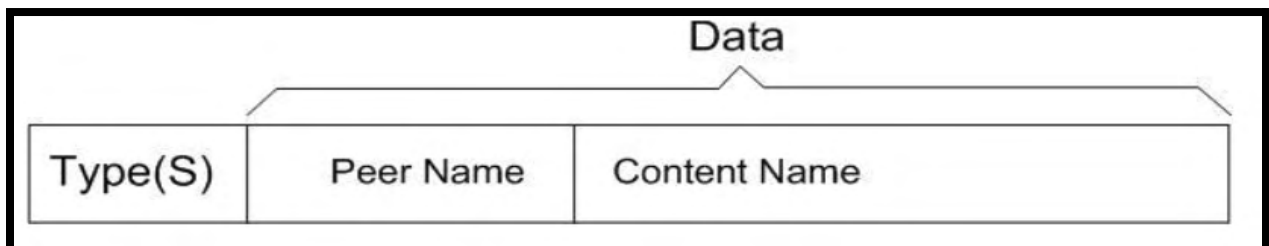


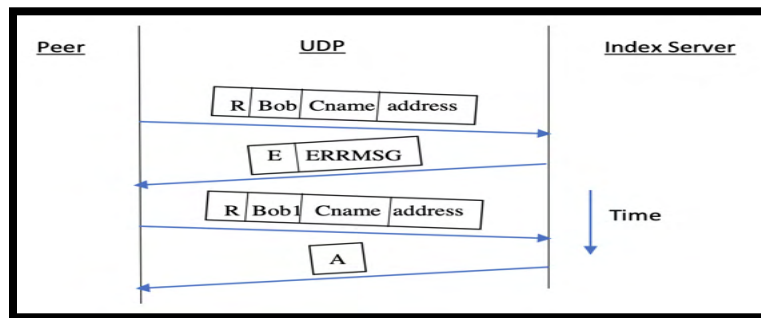
Figure 2.4: PDU - Search for Content and the Associated Content Server



III. Content Transfer - (List/Download)

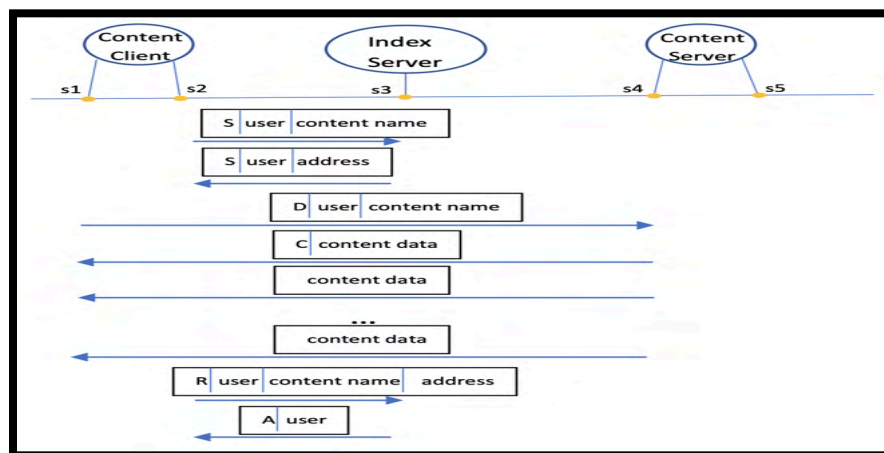
Transferring the file content between the Content Client and the Content Server via the established TCP connection is the protocol's last step. Two particular Protocol Data Units (PDUs) are used in this process: the Content Data (C-type) PDU and the Download Request (D-type) PDU.

Figure 2.5: Content Data PDU (O-type) Format



A Content Client uses a Download Request PDU to start the file transfer after successfully locating a Content Server via the Index Server and establishing a dedicated TCP connection.

Figure 2.6: User Datagram Protocol - Download Request from the PDU (D-type)



The actual file data is encapsulated and sent to the client by the Content Server using the C-type PDU. This PDU is special because, in order to contain the file content, its Data field size is not restricted to 100 bytes.

- **Segmentation:** The Content Server is required to segment the file because the content size may exceed the maximum TCP segment size. Several `write()` calls to the socket are used to stream the file across several C-type PDUs until the full file is transferred.
- **Header:** The receiving client can anticipate the amount of incoming data and control buffer allocation thanks to the important 4-byte header that specifies the total size of the content.

PROGRAMS

The following particular protocol operations, which are implemented using either UDP or TCP sockets, are made possible by the custom PDU format.

1. Index - Server Interaction (UDP-based Communication)

The Index Server's main responsibility is to oversee the metadata of the P2P network, offering services for content searches, deregistration, and registration. UDP sockets are used for all interactions with the Index Server, taking advantage of connectionless communication for control messages.

Content Registration (R-type PDU): To handle subsequent downloads, a Peer (Content Server) must first open a TCP passive socket (using `socket()`, `bind()`, and `listen()`). The unique Peer Name, the Content Name, and the server address (IP and TCP port) are then sent in an R-type PDU to the Index Server. The server responds with either an A-type PDU (Acknowledgment of success) or an E-type PDU (Error, urging the peer to choose a new name) after checking for conflicts (same Peer Name registering the same content).

Content Search (S-type PDU): To indicate the required Content Name, a Peer (Content Client) transmits an S-type PDU to the Index Server.

- To guarantee a uniform distribution of download traffic, the Index Server employs a load-balancing policy, choosing the content server that has been utilized the least for that content.
- In response, the Index Server sends either an E-type PDU if the content is not available or an S-type PDU with the address of the content server.

getsocketname system call

A peer that registers as a content server creates a socket that other peers can use to download the material. During registration, the index server must get the address that the socket is bound to. The IP address and port number make up the address. The peer just needs the port number information because the IP address is known (it links to the computer the peer is executing on). The user assigns the server's port number in each of the earlier labs. It is more reasonable to allow the OS's TCP module to choose the port number dynamically in this project. The code to accomplish that is as follows.

```
struct sockaddr_in reg_addr
s = socket(AF_INET, SOCK_STREAM, 0)
reg_addr.sin_family = AF_INET;
reg_addr.sin_port = htons(0);
reg_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
bind(s, (struct sockaddr *)&reg_addr, sizeof(reg_addr))
```

By assigning the port number to 0 in line 4, the peer requests the TCP module to choose a unique port number. To obtain the address, the peer can call *getsockname*.

```
alen = sizeof (struct sockaddr_in);
```

```
getsockname(p_sock, (struct sockaddr *) &reg_addr, &alen);
```

The port number is stored in *reg_addr.sin_port*; the IP address in *reg_addr.sin_addr.s_addr*. (You probably find the IP address stored in *reg_addr* is just 0.0.0.0, which means “this machine”, so this information is not very useful.)

Requesting and receiving a list of all registered content is done via Content Listing (O-type PDU). To delete records from the content registry, use Content De-Registration (T-type PDU). To ensure the integrity of the Index Server's records, peers must transmit a sequence of T-type PDUs to de-register all of their hosted material before departing.

2. Content Download (TCP-based Communication)

Using a dependable, connection-oriented TCP socket that is created using the address obtained from the Index Server, the actual content transmission takes place directly between peers.

Select System Call

From the discussion so far, it is obvious that the peer is attached to multiple sockets. As a content server, the peer that provides downloads for multiple contents needs to listen to multiple TCP sockets. In addition, it needs to listen to the stdin in order to interact with the user. The read and accept system calls can only listen to one socket at a time; thus, they alone are not enough to support the mechanism required by the peer.

To listen to multiple sockets, instead, the peer uses select. The system call *select* blocks the return, similar to *read* and *accept*, until one or more events (usually associated with pending TCP connection or data arrival) happen at the sockets it listens to. After the return, the peer can find out which socket(s) should be serviced and call *read* and/or *accept* to handle the event(s). The code below gives an example on how to setup the arguments before calling *select*.

```
fd_set rfd, afd;
```

```
sock = socket(AF_INET, SOCK_STREAM, 0);
```

```
FD_ZERO(&afd);
```

```
FD_SET(sock, &afd); /* Listening on a TCP socket */
```

```
FD_SET(0, &afd); /* Listening on stdin */
```



```
memcpy(&rfd, &afd, sizeof(rfd));
select(FD_SETSIZE, &rfd, NULL, NULL, NULL)
```

The type `fd_set` is a bit array. On line 4, if the argument `sock`, which is the descriptor of a TCP socket, has the value of 3, then `FD_SET` sets the 3rd bit of `afds` to 1; similarly, `FD_SET` sets the 0th bit of `afds` to 1 on line 5. When *select* is called on line 7, the return will be blocked until a TCP connection is pending at the TCP socket and/or data arrives at `stdin`. Upon the return of *select*, `rfd` indicates which socket (sockets) is (are) required for service. The following code shows a typical way to handle the return of *select*.

```
if (FD_ISSET(0, &rfd)) {
    n = read(0, buf, BUFSIZE);
    .
    .
    .
}
if(FD_ISSET(sock, &rfd)){
    new_sd = accept(sock, (struct sockaddr *)&client, &alen);
    .
    .
    .
} ...
```

If the first bit (0th position) of `rfd` is set, it implies that data has arrived at `stdin`; similarly, if the 4th bit (3rd position) of `rfd` is set, it implies there is a pending TCP connection. To handle the first case, a *read* is called to receive the data. For the second case, *accept* is called to complete the TCP connection.

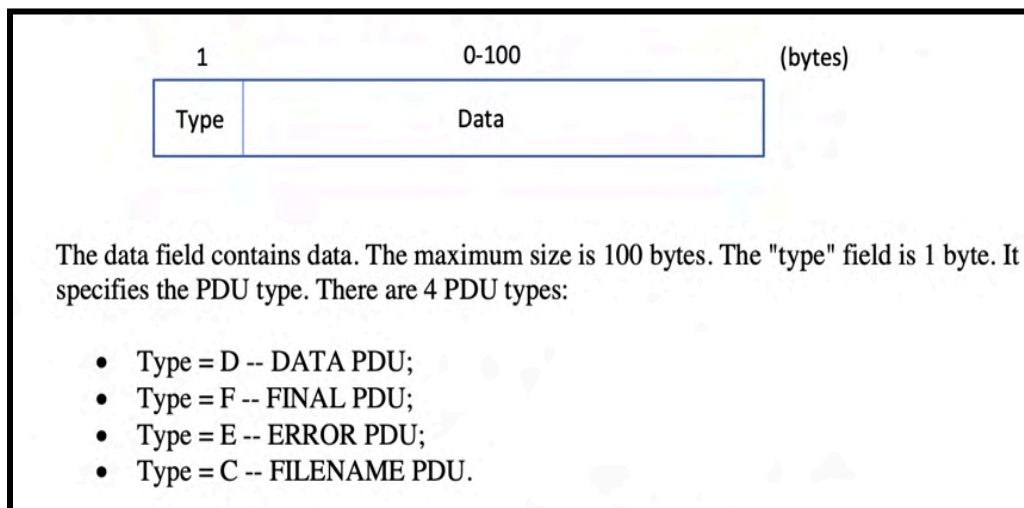
- I. **Connection Setup and Request:** Using `connect()`, the Content Client creates a TCP connection with the Content Server, initiating the TCP three-way handshake. After connecting, the client starts the transfer by sending a D-type PDU with the Content Name.
- II. **Content Transfer (C-type PDU):** Using one or more C-type PDUs, the Content Server finds the file and streams it to the client. Large files are handled by segmenting the information and delivering it across many TCP packets since files can exceed the maximum transmission unit (MTU). This method is implemented via consecutive `write()` operations to the socket.
- III. **Transfer Completion:** After transferring the final byte of the content, the Content Server explicitly ends the TCP connection (using the TCP four-way handshake) to inform the client that the download is finished.

- IV. **Self-Registration:** The P2P network is strengthened when the Content Client automatically registers with the Index Server as a new Content Server for that file following a successful download.

3. Content Listing

A peer should be able to de-register the material via the application. In order to deregister the content, the peer sends a T-type PDU to the index server. The index server will return an ack (A-type PDU) following a successful deregistration.

Figure 3.1: User Datagram Protocol (UDP) type and data structure



An artifact of using the connect() function on a UDP socket is the purpose of the first, ignored message. Because UDP is a connectionless protocol, the client can use simpler I/O calls like write() and read() rather than sendto() and recvfrom() because calling connect(sd,...) fixes the destination address (the server's address) for all subsequent send and receive operations on that socket rather than establishing a handshake. A zero-length or minimum UDP packet is sent to the selected destination on certain operating system implementations upon the initial attempt to use the "connected" UDP socket with a data function such as write() or read(). The OS uses this packet for internal network stack functions (such as route resolution), but it lacks a legitimate application-layer PDU. As a result, the server appropriately ignores it, expecting only a well-formed PDU with the file request type 'C'.

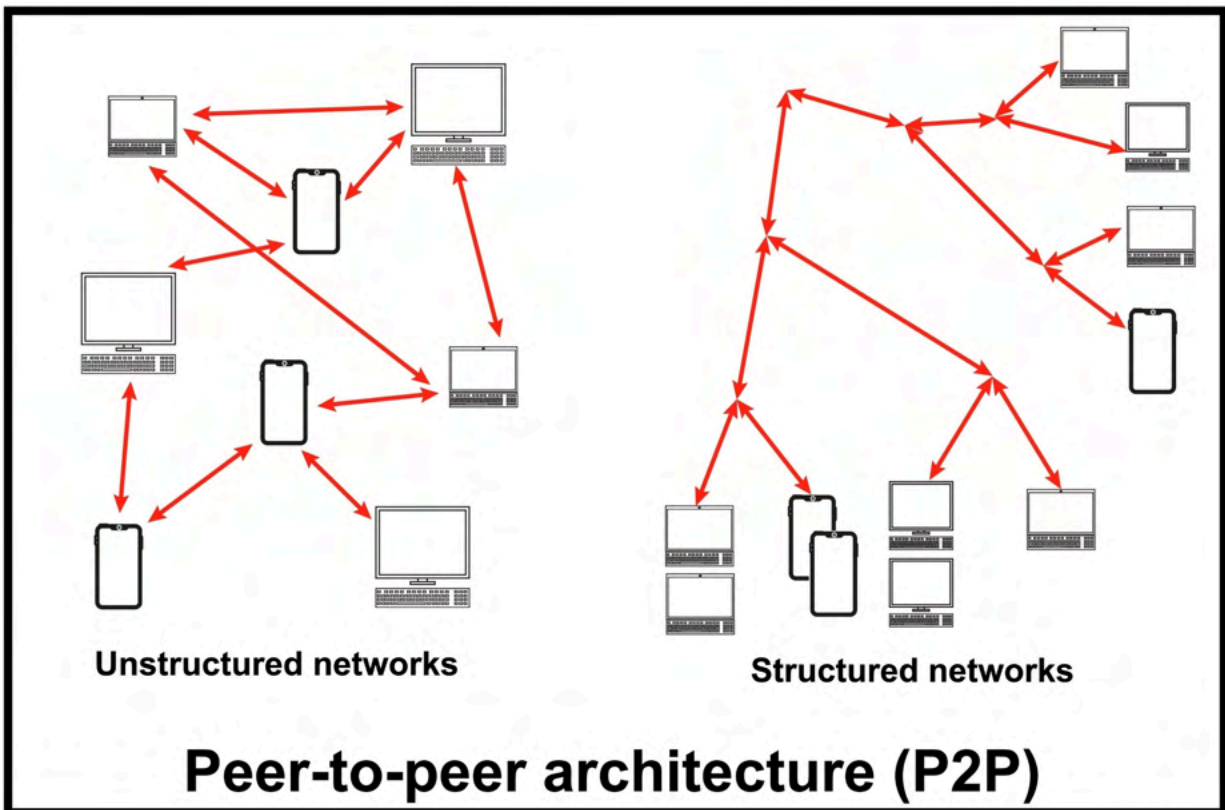
4. Content De-Registration

A peer should be able to de-register the material via the application. In order to deregister the content, the peer sends a T-type PDU to the index server. The index server will return an ack (A-type PDU) following a successful deregistration.

5. Quit

Every time a peer wishes to quit, it must first de-register all of its registered content in order to maintain the content registration data at the index server current. Sending a sequence of T-type PDUs to the index server prior to quitting can accomplish this. One content will be de-registered by each T-type PDU.

Image 3.2: P2P Mechanism Flow

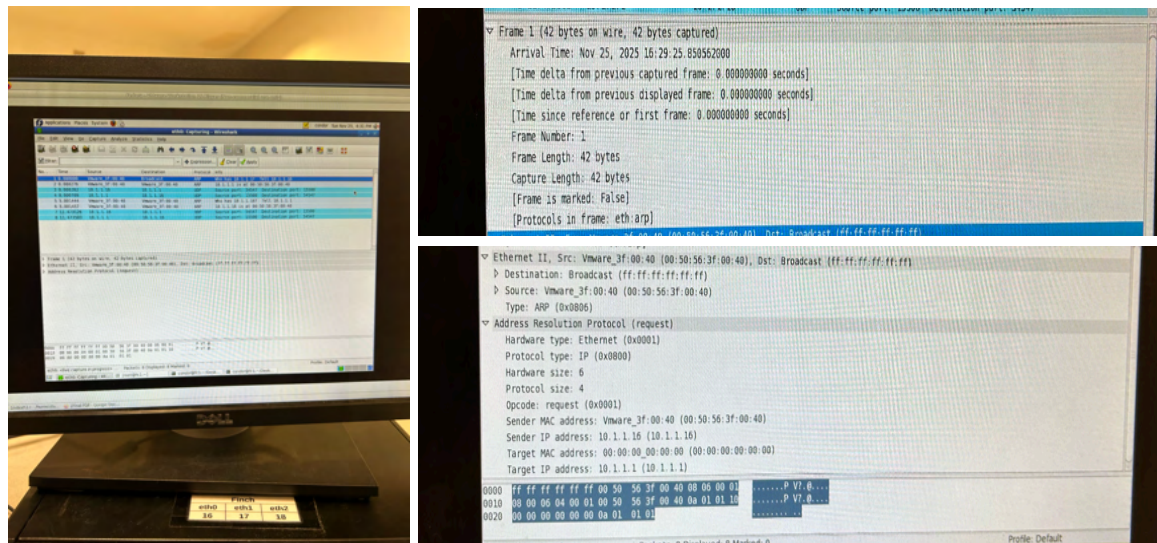


OBSERVATIONS & ANALYSIS

The following steps comprise the main mechanism for content sharing:

1. A peer registers its content, along with its IP address and a listening TCP port, with the Index Server while functioning as a content server.
2. In order to determine the address of a content server for the desired file, a peer, acting as a content client, asks the Index Server.
3. The client connects directly to the content server to download the file.
4. The client immediately registers as a new content server for that content after a download is finished, strengthening the P2P network's decentralized structure and scalability.

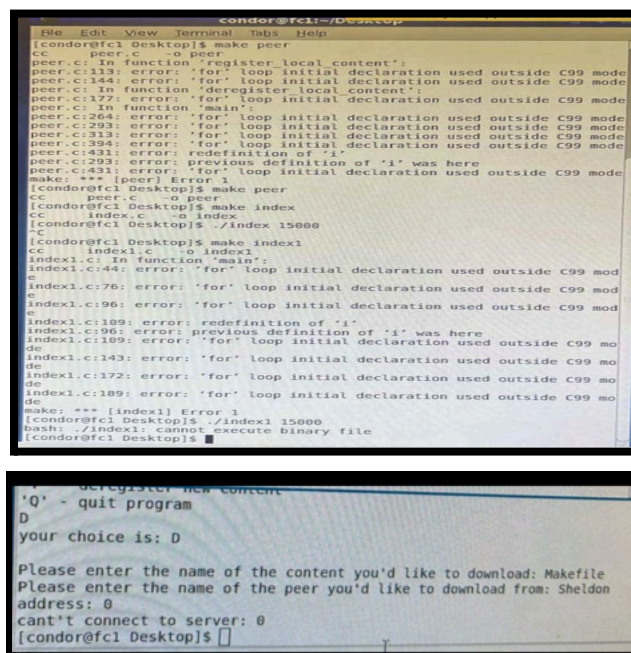
Image 4.1: Configuration of Ethernet protocol in the network | server (index) - peer (ports)



A straightforward PDU used to delete content entries from the Index Server's registry is depicted in the figure showing the Deregistration PDU structure.

- **Function:** The withdrawal of content availability is signalled by the F-type PDU. When a peer exits the application, it is required by Demonstration Requirement 1.f to send an F-type PDU for each piece of registered content.
- **Server Use:** To prevent clients from being sent to an inaccessible server, the Index Server locates and deletes the related Peer's address from its content registry for that file using the Content Name contained in the PDU.

Figure 4.2: PDU used bidirectionally to facilitate content location and peer address provisioning



A Peer (Content Client) and the Index Server can communicate across an unreliable UDP channel thanks to the Search PDU (S-type), a control message used for content discovery. Both the client's request and the Index Server's answer regarding the placement of the content are supported by the PDU's structure.

- Search Request: The S-type containing the required Content Name is sent to the Index Server by the Content Client.
- Search Response: The client can create a dedicated TCP connection for the next file download when the Index Server responds with the S-type, filling in the Peer Address field with the coordinates of the chosen Content Server.

The Index Server's current registration of all accessible content and the Peers' (Content Servers') hosting of it are essential to the P2P application's successful operation. The Registration PDU (R-type) and Deregistration PDU (F-type), two specific UDP-based PDUs, are used to accomplish this crucial content management.

Figure 4.3: User Datagram Protocol (UDP) of sending a frame as a socket[sd]

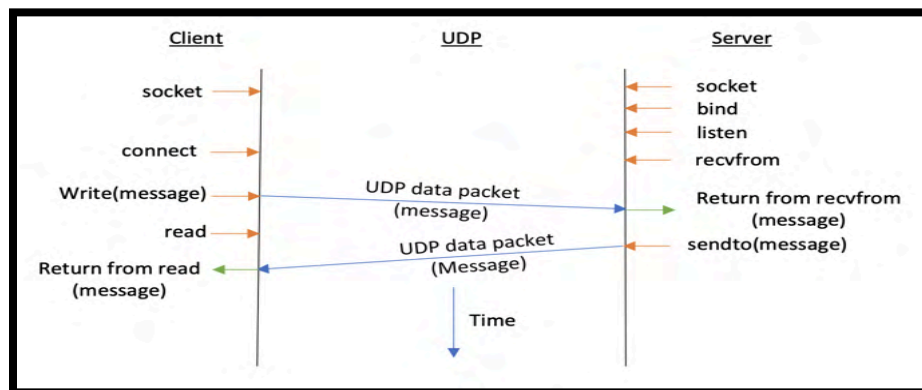


Figure 4.4: This PDU is sent by a Peer to the Index Server, providing both the Content Name and the Peer Address (IP and TCP port) where the file can be downloaded & Content Deregistration PDU (F-type) Format

```

condor@fcl:~/Desktop
File Edit View Terminal Tabs Help
^C
[condor@fcl Desktop]$ ./peer 10.1.1.25 5000
Enter Peer Name:
Maisam

Please select one of the following actions to complete:
'R' - register new content
'g' - get list of content
'd' - download new content
'S' - search for content
'T' - deregister new content
'Q' - quit program

[condor@fcl Desktop]$ ./index 5050
The peer wants to register new data
peer name: Maisam
content name: Makefile
address: 0
port: 46480
Online content request
Search content request
peer name: Sheldon
content name: Makefile

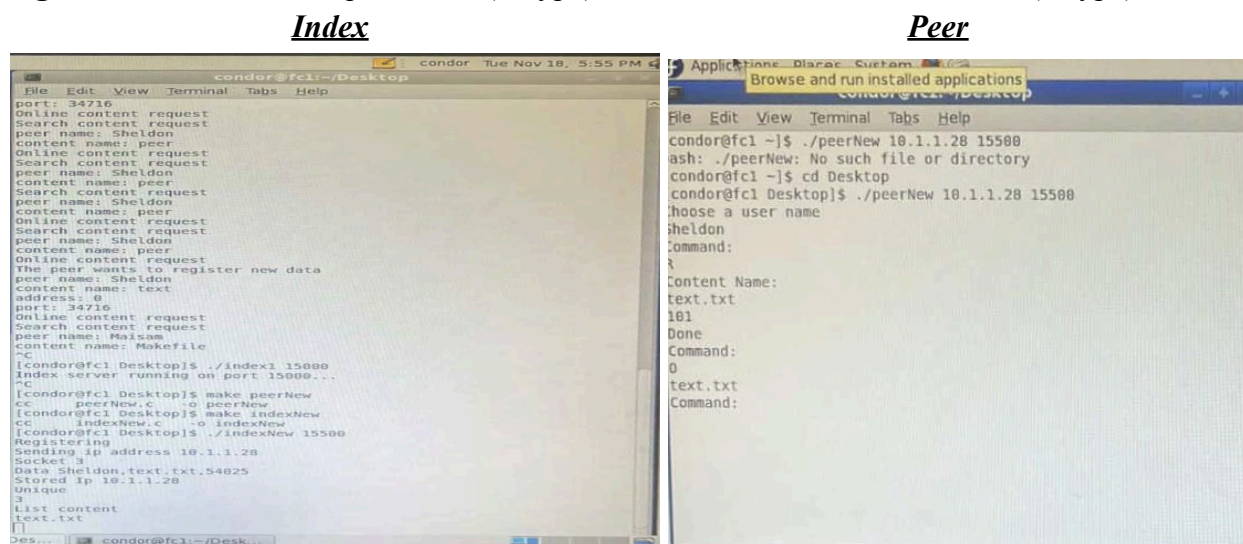
```


The elements required for a peer to declare itself as a content server for a particular file are depicted in the Registration PDU diagram.

- **Function:** To register a piece of content, a peer sends the Index Server an R-type PDU. According to Demonstration Requirement 1.d, this must happen automatically once a peer successfully downloads a file and during program launch for files that were first hosted.
- **Server Use:** The Index Server associates the file name with the network address of the hosting peer by using the data in this PDU to update its content registry.

In order to delete a particular Content Name item from the registry, a Peer sends this PDU to the Index Server. This usually happens when the Peer shuts down or ceases serving the file in some other way.

Figure 4.5: Download Request PDU (D-type) Format & Get Content List PDU (L-type)



The message format used to start the file transfer process between a Content Client and a Content Server is depicted in the Download Request PDU picture.

- **Function:** After the connection is made using the address found in the Search PDU, the D-type PDU is the first message sent over the TCP connection between the Content Client and the Content Server. It indicates which particular content the client wants to download.
- **Server Use:** The Content Server confirms the Content Name after receiving the D-type communication. The server sends the file's contents over the designated TCP socket if the file is present.

The control message used to ask the Index Server for a directory of accessible material is depicted in the get material List PDU picture.

- **Function:** Any Peer that wants a comprehensive list of all content that is currently registered across all Content Servers sends the L-type PDU to the Index Server. This talks about Requirement for Demonstration 1.
- **Index Server:** provides a list of the registered content names to the peer. The response mechanism, which is not depicted in this PDU form, must support a variable-length list of 10-byte content names; this PDU is only for the request.

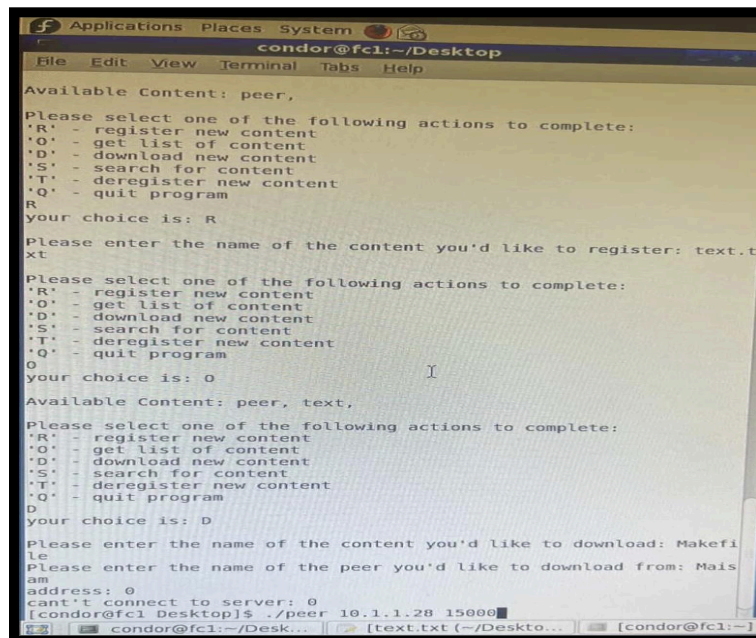
Figure 4.6: Core functionality of the P2P (Peer-to-Peer) application

<u>Server</u>	<u>Peer</u>
<pre> condor@fcl:~/Desktop File Edit View Terminal Tabs Help index1.c:76: error: 'for' loop initial declaration used outside C99 mod e index1.c:96: error: 'for' loop initial declaration used outside C99 mod e index1.c:109: error: redefinition of 'i' index1.c:96: error: previous definition of 'i' was here index1.c:109: error: 'for' loop initial declaration used outside C99 mo de index1.c:143: error: 'for' loop initial declaration used outside C99 mo de index1.c:172: error: 'for' loop initial declaration used outside C99 mo de index1.c:189: error: 'for' loop initial declaration used outside C99 mo de make: *** [index1] Error 1 [condor@fcl Desktop]\$./index1 15000 bash: ./index1: cannot execute binary file [condor@fcl Desktop]\$ make index cc index.c -o index [condor@fcl Desktop]\$ make peer cc peer.c -o peer [condor@fcl Desktop]\$./index 15000 ^C [condor@fcl Desktop]\$./index 15000 The peer wants to register new data peer name: Sheldon content name: peer address: 0 port: 34716 Online content request Search content request peer name: Sheldon content name: peer Online content request Search content request peer name: Sheldon content name: peer Search content request peer name: Sheldon content name: peer Online content request Search content request peer name: Sheldon content name: peer </pre>	<pre> condor@fcl:~/Desktop File Edit View Terminal Tabs Help 'D' - get list of content 'D' - download new content 'S' - search for content 'T' - deregister new content 'Q' - quit program 0 your choice is: 0 Please enter the name of the content you'd like to download: peer Please enter the name of the peer you'd like to download from: Shel don address: 0 cant't connect to server: 0 [condor@fcl Desktop]\$./peer 10.1.1.20 15000 Enter Peer Name: Maisam Please select one of the following actions to complete: 'R' - register new content 'D' - get list of content 'D' - download new content 'S' - search for content 'T' - deregister new content 'Q' - quit program 0 your choice is: 0 Available Content: peer, Please select one of the following actions to complete: 'R' - register new content 'D' - get list of content 'D' - download new content 'S' - search for content 'T' - deregister new content 'Q' - quit program 0 your choice is: 0 Please enter the name of the content you'd like to download: peer Please enter the name of the peer you'd like to download from: Shel don address: 0 cant't connect to server: 0 [condor@fcl Desktop]\$ </pre>

The three primary stages of the peer-to-peer (P2P) content exchange system require the usage of TCP for direct file transfers between peers and UDP for control communication with the Index Server. The initial Content Registration stage, in which a peer registers the file and its address with the Index Server, is confirmed by displaying the Index Server's results.

1. Sends a UDP search request and obtains the address of the content server.
2. Successfully establishes a direct TCP connection and retrieves the file using this address, completing the Content Download.
3. The process ends with the automatic De-registration phase, in which the downloading peer instantly registers as a new content server for its own address. This increases the file's availability in the P2P environment and strengthens the network's resilience.

Figure 4.7: Application Menu



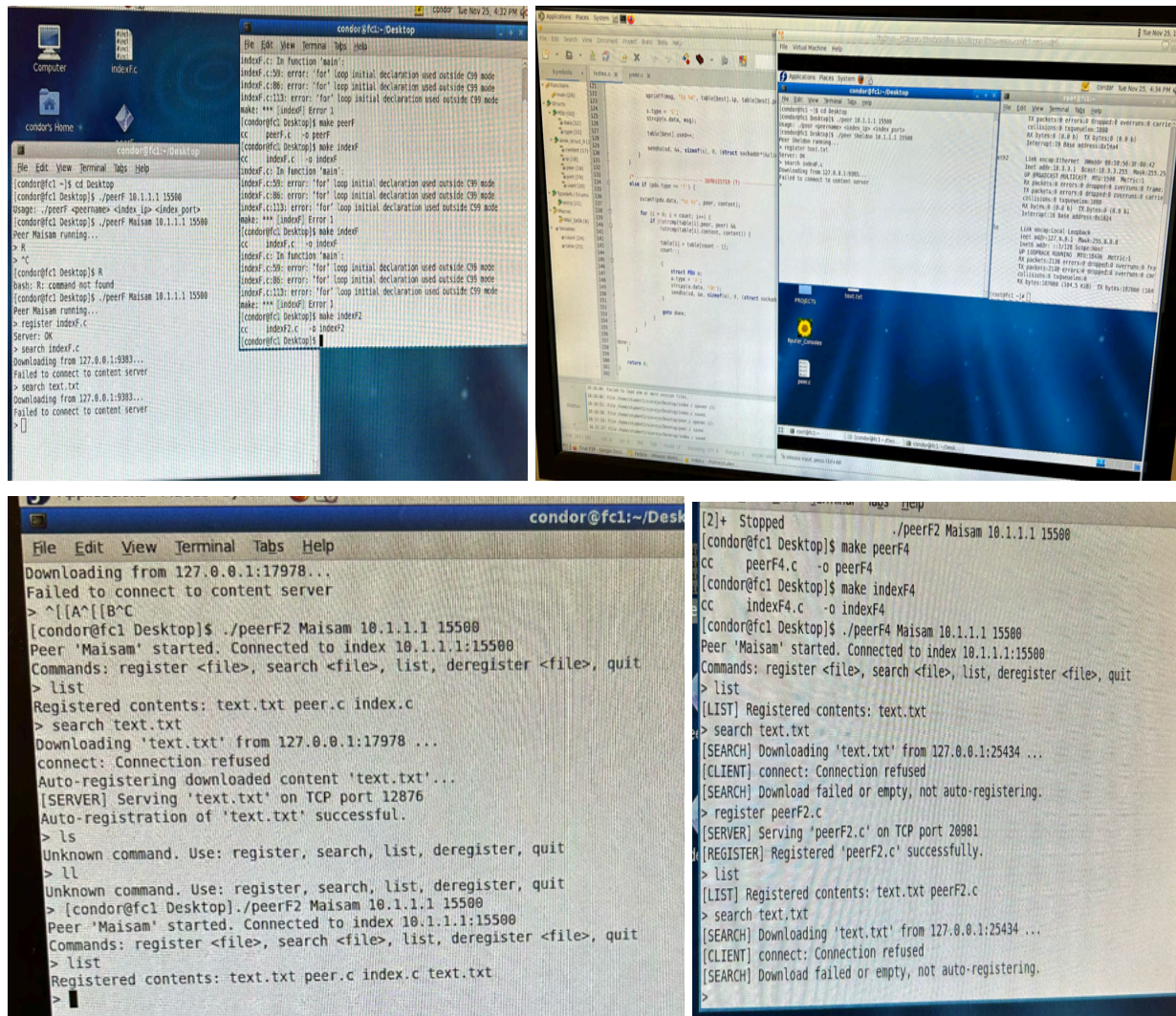
```
Applications Places System
condor@fcl:~/Desktop
File Edit View Terminal Tabs Help

Available Content: peer,
Please select one of the following actions to complete:
'R' - register new content
'O' - get list of content
'D' - download new content
'S' - search for content
'T' - deregister new content
'Q' - quit program
R
your choice is: R
Please enter the name of the content you'd like to register: text.txt
Please select one of the following actions to complete:
'R' - register new content
'O' - get list of content
'D' - download new content
'S' - search for content
'T' - deregister new content
'Q' - quit program
O
your choice is: O
Available Content: peer, text,
Please select one of the following actions to complete:
'R' - register new content
'O' - get list of content
'D' - download new content
'S' - search for content
'T' - deregister new content
'Q' - quit program
D
your choice is: D
Please enter the name of the content you'd like to download: Makefile
Please enter the name of the peer you'd like to download from: Maisam
address: 0
cant't connect to server: 0
[condor@fcl Desktop]$ ./peer 10.1.1.28 15000
[condor@fcl Desktop]$
```

- (1) Registration of Content: As seen in the screenshot with the "REGISTER" command and success message, Peer 1, the initial content server, sends a UDP packet to the Index Server in order to register the material it contains. The Index Server records Peer 1's address and content identification and then
- (2) sends a UDP message to Peer 1 acknowledging the registration (as shown in the server's output).
- (3) The peer command line that displays "SEARCH" indicates that Peer 2 then initiates a Content Search by sending the Index Server a UDP query for the content. The Index Server examines its database to find the content.
- (4) replies with a UDP message with the network address of the content server (Peer 1). Peer 2 then establishes a way to avoid using the Index Server for the actual data transfer.
- (5) Use this address to send a TCP Connection Request straight to Peer 1.
- (6) When Peer 1 accepts the connection, Peer 2 uses TCP to download the content from Peer 1; the peer's command output often reports the success and progress of this procedure. When the download is successful, Peer 2 immediately moves to a different content server for the same file.
- (7) Using UDP to register the material with the Index Server.
- (8) Terminate the TCP connection used to carry out the download. This approach demonstrates a basic P2P principle: peers who receive content automatically become new content servers, ensuring content resilience and diffusion. The many screenshots, which show peer commands for content search and registration as well as server reply confirming registration and content server information, validate these stages.

Automation & Authentication (PDU-based communication)

Figure 4.8: Sequential algorithm of the successful Protocol Datagram Unit (PDU)



The P2P program relies on an Index Server to facilitate peer-to-peer connections using a two-tiered protocol approach: TCP for the actual content transfer and UDP for index-related communications.

- How the server transfers the complete file to the client.

By reading the file in 100-byte chunks (specified by BUFLen) and encapsulating each chunk into a unique PDU (Protocol Data Unit), the server sends the entire file to the client. The server uses `fread()` in a loop to read the file segment by segment. It sets the PDU's type field to 'D' (Data) for each segment. The server signals the conclusion of the file by setting the PDU type to 'F' (Final) after reading the last segment. The connectionless `sendto()` function is then used to send each PDU, which includes the type byte and the file data, to the client.

- How the client receives the complete file.

Through a continuous loop that performs the read() method on the UDP socket, the client receives the whole file contained in the receiveFile function. Each received PDU's type field is examined by the client. If the type is 'D', fwrite() is used to write the data payload to the local file. The client instantly ends the reception loop if the type is 'F', in which case the final data payload is written to the file. When the final PDU is received, this technique guarantees that the client processes every segment and correctly ends the file download.

- How the client handles the error message.

In order to handle error messages, the client uses the receiveFile function to look for an error PDU of type 'E'. The client publishes the precise error message found in the PDU's data field to standard error if an error PDU is received. Importantly, in order to avoid a corrupt download from staying on the system, the client does a cleanup by closing the file and using remove() to remove the partially formed local file. The function then indicates to the main() calling function that the file receiving was not successful by returning a non-zero result.

CONCLUSIONS

The successful implementation of the P2P content sharing system validated a hybrid communication architecture that makes use of both TCP and UDP protocols. By strategically utilizing an Index Server for centralized metadata management (registration and discovery) and direct peer-to-peer TCP connections for dependable content transmission, the fundamental goal of building a robust, decentralized network for content exchange was achieved. The effective management of concurrent I/O utilizing the select system call, which enables a single process to concurrently perform the functions of server, client, and user interface handler, was essential to the Peer component's success.

Several significant outcomes from the design and implementation stages support the project's guiding principles: Using a two-tiered protocol stack turned out to be the best choice in terms of dependability and efficiency. All Index Server interactions, including Content Registration and Search (R-type and S-type PDUs), were successfully handled by the Control Plane (UDP). The fast, low-overhead signalling required for quick discovery services was guaranteed by this connectionless method. In order to ensure the connection-oriented reliability needed for the precise transfer of huge file segments between the Content Client and Content Server, the Data Plane (TCP) was required for Content Download.

A uniform communication standard was created by the unique application-layer protocol, which was specified by the straightforward PDU structure (1-byte Type Field + Data Field). All control and data messages (such as Download Request D-type and Content Data C-type) were processed consistently across the various transport protocols thanks to this abstraction, which made message construction and parsing easier.

The technical difficulty of combining several concurrent network responsibilities into a single peer process was effectively resolved by the project. The select system call was needed for

non-blocking I/O. The Peer might effectively reply to user commands, pending download requests, and Index Server replies without dedicated threads by keeping an eye on the stdin, passive TCP socket, and active TCP/UDP sockets. This would preserve system resources. The Peer had a reliable way to retrieve a dynamically assigned TCP port from the OS by using bind with port 0 and then getsockname(). This address was accurately transmitted to the Index Server during registration, ensuring that other Peers could correctly establish direct connections for content download.

The foundation of the P2P aspect of the system is the self-registration procedure that follows a successful download. A client becomes a new content source by instantly registering the newly obtained content with the Index Server. This distributes the hosting load and increases the content's general availability and network diffusion. Additionally, the required Deregistration (T-type PDU sequence) prior to program exit guarantees the accuracy of the Index Server's registry and keeps clients from being routed to unavailable content servers.

The foundation of the P2P aspect of the system is the self-registration procedure that follows a successful download. A client becomes a new content source by instantly registering the newly obtained content with the Index Server. This distributes the hosting load and increases the content's general availability and network diffusion. Additionally, the required Deregistration (T-type PDU sequence) prior to program exit guarantees the accuracy of the Index Server's registry and keeps clients from being routed to unavailable content servers.

APPENDIX

peer.c

```
// COE768 - Design Project
// November 25th, 2025

#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
// Structure of PDU
struct PDU {
    char type;
    char data[100];
};
// Definitions
#define BUFSIZE sizeof(struct PDU)
#define ERROR      'E'
#define REGISTER 'R'
#define ONLINE     'O'
#define QUIT      'Q'
#define SEARCH    'S'
#define ACKNOWLEDGEMENT 'A'
#define DEREG     'T'
#define CONTENT 'C'
#define DOWNLOAD 'D'

// Universal variables
int      s, n, type, new_tcp_socket;
char peer_name[10];

// Used to print off all the selection options each iteration
void printOptions(){
    printf("\nPlease select one of the following actions to complete: \n");
    printf("'R' - register new content\n");
    printf("'O' - get list of content\n");
    printf("'D' - download new content\n");
    printf("'S' - search for content\n");
    printf("'T' - deregister new content\n");
    printf("'Q' - quit program\n");
}

// Used to register new content
void registerContent(char content[], char peer[]){
```

```

// Check and make sure the file you're offering exists on the computer
if (access(content, F_OK) != 0){
    printf("This file does not exist");
    return;
}
// Make tcp connection
int    tcp_socket, new_sd, client_len, port, alen, message_read_size;
struct sockaddr_in server, client;
struct PDU msg, r_msg;
char   tcp_address[5], tcp_port[6], send_data[100];

if ((tcp_socket = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
    fprintf(stderr, "Can't create a socket\n");
    exit(1);
}

bzero((char *)&server, sizeof(struct sockaddr_in));
server.sin_family = AF_INET;
server.sin_port = htons(0);
server.sin_addr.s_addr = htonl(INADDR_ANY);
if (bind(tcp_socket, (struct sockaddr *)&server, sizeof(server)) == -1){
    fprintf(stderr, "Can't bind name to socket\n");
    exit(1);
}
alen = sizeof(struct sockaddr_in);
getsockname(tcp_socket, (struct sockaddr *)&server, &alen);

// Save the address and port number of the tcp socket
snprintf(tcp_address, sizeof(tcp_address), "%d", server.sin_addr.s_addr);
snprintf(tcp_port, sizeof(tcp_port), "%d", htons(server.sin_port));

// Make R type message to send to the index
memset(&msg, '\0', sizeof(msg));
msg.type = REGISTER;

strcpy(send_data, peer);
strcat(send_data, ";");
strcat(send_data, content);
strcat(send_data, ";");
strcat(send_data, tcp_address);
strcat(send_data, ";");
strcat(send_data, tcp_port);
strcpy(msg.data, send_data);

// Send message to index
write(s, &msg, sizeof(msg));

// Reads response from index
memset(&r_msg, '\0', sizeof(r_msg));
read(s, &r_msg, sizeof(r_msg));

```

```

// If response from index is error, don't register content
if (r_msg.type == ERROR){
    //handle an error
    printf("The data cant be registered");
}
// If response from index is acknowledgement, register content
else if (r_msg.type == ACKNOWLEDGEMENT){
    // Clear variables
    memset(&msg, '\0', sizeof(msg));
    memset(&r_msg, '\0', sizeof(r_msg));
    // Split into child and parent to host content
    listen(tcp_socket, 5);
    switch (fork()) {
    case 0:
        while (1){
            // Set up new branch
            int client_length = sizeof(client);
            new_tcp_socket = accept(tcp_socket, (struct sockaddr
*)&client, &client_length);

            // Listen to message from other peer
            read(new_tcp_socket, &r_msg, sizeof(msg));
            // If message from peer is type 'D', start download process
            if (r_msg.type == DOWNLOAD){
                // Open up the file
                FILE *file;
                file = fopen(content, "rb");
                if (!file){
                    printf("This file doesn't exist\n");
                }
                // If the file exists, iteratively send messages to peer
                else {
                    msg.type = CONTENT;
                    while (fgets(msg.data, sizeof(msg.data),
file)>0){
                        write(new_tcp_socket, &msg,
sizeof(msg));
                    }
                }
                // Close the file
                fclose(file);
                close(new_tcp_socket);
                break;
            }
            // If the message from the other peer isn't 'D' type
            else {
                break;
            }
        }
    }
}

```



```

        break;
    case -1:
        fprintf(stderr, "fork: error\n");
        break;
    }
}

// Used to download content from another peer
void downloadContent(char content_name[], char address[]){
    //variables
    int d_socket, data, content_counter=0;
    struct sockaddr_in server;
    struct PDU msg, r_msg;
    struct hostent *host;
    char *serverHost = "0";

    //create TCP socket
    if ((d_socket = socket(AF_INET, SOCK_STREAM, 0))<0){
        fprintf(stderr, "Can't create a TCP socket\n");
        exit(1);
    }

    bzero((char *)&server, sizeof(struct sockaddr_in));
    server.sin_family = AF_INET;
    server.sin_port = htons(atoi(address));
    printf("address: %d\n", atoi(address));
    if (host = gethostbyname(serverHost)){
        bcopy(host->h_addr, (char *)&server.sin_addr, host->h_length);
    }
    else if (inet_aton(serverHost, (struct in_addr *)&server.sin_addr)){
        fprintf(stderr, "Can't get server's address\n");
    }

    if (connect(d_socket, (struct sockaddr *)&server, sizeof(server)) == -1){
        fprintf(stderr, "cant't connect to server: %s\n", host->h_name);
    }

    // Send download request to peer
    msg.type = DOWNLOAD;
    strcpy(msg.data, peer_name);
    strcat(msg.data, ";");
    strcat(msg.data, content_name);
    write(d_socket, &msg, sizeof(msg));

    // Open a new file to store downloaded content
    FILE *file;
    file = fopen("tester", "w+"); //change this after testing
    //Iteratively get data from the peer and add it to the new file
    while ((data = read(d_socket, &r_msg, sizeof(r_msg)))>0){
        if(r_msg.type == CONTENT){

```

```

        fprintf(file, "%s", r_msg.data);
        content_counter += 1;
    }
    else if (r_msg.type == ERROR){
        printf("error message recieved");
    }
}
fclose(file);
// If content was downloaded, register it
if (content_counter > 0){
    registerContent(content_name, peer_name);
}
}
// Main part of the program
int main(int argc, char **argv)
{
    char    *host = "localhost";
    int     port = 3000;
    char    now[100];
    struct hostent *phe;
    struct sockaddr_in sin;

    int     socket_read_size;
    char    socket_read_buf[BUFSIZE];
    int     c_read_size;
    char    c_read_buf[100];
    struct PDU msg, r_msg;
    FILE*   fp;
    char    file_name[100], data_send[100];
    int     error_found = 0;
    char    test_file_name[100] = "", choice[1], content_name[10], address[80]="placeholder
address", input[10], input2[10];
    char    temp_port[10], temp_addr[10], temp_content_name[10], temp_peer_name[10];

    // Set up the UDP socket
    switch (argc) {
        case 1:
            break;
        case 2:
            host = argv[1];
        case 3:
            host = argv[1];
            port = atoi(argv[2]);
            break;
        default:
            fprintf(stderr, "usage: UDP File Download [host [port]]\n");
            exit(1);
    }

    memset(&sin, 0, sizeof(sin));

```

```

sin.sin_family = AF_INET;
sin.sin_port = htons(port);

if ( phe = gethostbyname(host) ){
    memcpy(&sin.sin_addr, phe->h_addr, phe->h_length);
}
else if ( (sin.sin_addr.s_addr = inet_addr(host)) == INADDR_NONE )
    fprintf(stderr, "Can't get host entry \n");

s = socket(AF_INET, SOCK_DGRAM, 0);
if (s < 0)
    fprintf(stderr, "Can't create socket \n");

if (connect(s, (struct sockaddr *)&sin, sizeof(sin)) < 0) {
    fprintf(stderr, "Can't connect to %s\n", host);
}

// Get the name of the peer on start up
memset(&peer_name, '\0', sizeof(peer_name));
printf("Enter Peer Name:\n");
c_read_size = read(0, peer_name, BUFSIZE);
if (c_read_size >= 10) {
    printf("ERROR\n");
    return -1;
}
peer_name[strlen(peer_name)-1] = '\0';

// Main loop of the program
while(1) {
    // Reset variables
    memset(&input, '\0', sizeof(input));
    memset(&msg, '\0', sizeof(msg));

    // Print Options
    printOptions();
    read(0, choice, 2);
    printf("your choice is: %s\n", choice);

    //Start of the options
    if (choice[0] == 'O') { //ONLINE CONTENT
        //Write request to index
        msg.type = ONLINE;
        strcpy(msg.data, "online content");
        write(s, &msg, BUFSIZE);
        //Recieve message
        read(s, &r_msg, BUFSIZE);
        printf("Available Content: ");
        printf("%s\n", r_msg.data);
    }
    else if (choice[0] == 'R') { //REGISTER CONTENT

```

```

        //send R-type message to index
        msg.type = REGISTER;
        printf("Please enter the name of the content you'd like to register: ");
        scanf("%9s", input);
        registerContent(input, peer_name);
    }
    else if (choice[0] == 'D') { //DOWNLOAD CONTENT
        printf("Please enter the name of the content you'd like to download: ");
        scanf("%9s", input);
        printf("Please enter the name of the peer you'd like to download from: ");
        scanf("%9s", input2);
        //send S-type to index
        msg.type = SEARCH;
        strcpy(msg.data, input2);
        strcat(msg.data, ";");
        strcat(msg.data, input);
        write(s, &msg, BUFSIZE);
        // Recieve message from index
        read(s, &r_msg, BUFSIZE);
        sscanf(r_msg.data, "%[a-zA-Z0-9]; %[a-zA-Z0-9] %[0-9]; %[0-9]",
temp_content_name, temp_peer_name, temp_addr, temp_port);
        // Download the content
        downloadContent(temp_content_name, temp_port);
    }
    else if (choice[0] == 'T') { //DEREGISTER CONTENT
        printf("Please enter the name of the content you'd like to deregister: ");
        scanf("%s", input);
        if (strcmp(input, "ntent")==0){
            strcpy(input, "content");
        }
        // Send deregister message to index
        msg.type = Dereg;
        printf("peer name: %s\n", peer_name);
        strcpy(msg.data, peer_name);
        strcat(msg.data, ";");
        strcat(msg.data, input);
        write(s, &msg, BUFSIZE);
        // Recieve message from index
        read(s, &r_msg, BUFSIZE);
        if (r_msg.type == ACKNOWLEDGEMENT){
            printf("Content was successfully deregistered\n");
        }
        else if (r_msg.type == ERROR){
            printf("Error: %s\n", r_msg.data);
        }
    }
    else if (choice[0] == 'S'){
        printf("Please enter the name of the content you'd like to search for: ");
        scanf("%9s", input);
        // Send search request to the index

```

```

        msg.type = SEARCH;
        strcpy(msg.data, peer_name);
        strcat(msg.data, ";");
        strcat(msg.data, input);
        write(s, &msg, BUFSIZE);
        read(s, &r_msg, BUFSIZE);
        sscanf(r_msg.data, "%[a-zA-Z0-9];%[a-zA-Z0-9];%[0-9];%[0-9]",
temp_content_name, temp_peer_name, temp_addr, temp_port);
        printf("Here is the information you searched for: \n");
        printf("Content Name: %s\n", temp_content_name);
        printf("Peer Name: %s\n", temp_peer_name);
        printf("Address: %s\n", temp_addr);
        printf("Port Number: %s\n", temp_port);
    }
    else if (choice[0] == 'Q') { //QUIT CONTENT
        // Send quit request to the index
        msg.type = QUIT;
        strcpy(msg.data, peer_name);
        write(s, &msg, BUFSIZE);
        // Read return message from index
        read(s, &r_msg, BUFSIZE);
        // If the return message is type 'Q', peer can quit
        if (r_msg.type == QUIT){
            break;
        }
        // If the return message is type 'E', peer can't quit
        else if (r_msg.type == ERROR){
            printf("%s\n", r_msg.data);
        }
    }
}
close(s);
exit(0);
}

```

```

// COE768 - Design Project
// November 11th, 2025

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/unistd.h>
#include <sys/signal.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <strings.h>
#include <netdb.h>
#include <stdio.h>
#include <time.h>

// Structure of the PDU
struct PDU {
    char type;
    char data[100];
};

// Structure to store registered content
typedef struct{
    char content[10];
    char peer[10];
    char address[10];
    char port_number[10];
} reg_data;

// Definitions
#define BUFSIZE sizeof(struct PDU)
#define ERROR      'E' //report error
#define ONLINE     'O'
#define QUIT       'Q'
#define REGISTER   'R'
#define DEREG      'T'
#define SEARCH     'S'
#define ACKNOWLEDGEMENT 'A'

// Main section //////////////////////////////////////
int main(int argc, char *argv[])
{
    // Variables
    struct sockaddr_in fsin;
    char buf[100];
    char *pts;
    int sock;
    time_t now;

```



```

int    alen;
struct sockaddr_in sin;
int    s, type, error_found = 0;
int    port = 3000;

struct PDU msg, send_msg;
int    socket_read_size, i = 0;
char    socket_buf[BUFSIZE];
char    file_name[100], online_content[100];
int    file_size, registered = 0, found = 0, placement = 0;
reg_data registered_content[10];
char data_recieved[100], peer_name[10], content_name[10], address[10], port_num[10];
int content_tracking[] = {0,0,0,0,0,0,0,0,0,0};

// Set up UDP server connection
switch(argc){
    case 1:
        break;
    case 2:
        port = atoi(argv[1]);
        break;
    default:
        fprintf(stderr, "Usage: %s [port]\n", argv[0]);
        exit(1);
}

memset(&sin, 0, sizeof(sin));
sin.sin_family = AF_INET;
sin.sin_addr.s_addr = INADDR_ANY;
sin.sin_port = htons(port);

/* Allocate a socket */
s = socket(AF_INET, SOCK_DGRAM, 0);
if (s < 0) {
    fprintf(stderr, "can't creat socket\n");
}

/* Bind the socket */
if (bind(s, (struct sockaddr *)&sin, sizeof(sin)) < 0) {
    fprintf(stderr, "can't bind to %d port\n", port);
}

listen(s, 5);
alen = sizeof(fsin);

// Main section, after UDP connection is set up
while (1) {
    // Reset variables
    memset(&send_msg, '\0', sizeof(send_msg));

```

```

        memset(&data_recieved, '\0', sizeof(data_recieved));
        registered = 0;
        found = 0;
        error_found = 0;

        // Get message from peer
        if (recvfrom(s, socket_buf, sizeof(socket_buf), 0, (struct sockaddr *)&fsin, &alen) < 0)
        {
            printf("recvfrom error\n");
        }
        // Copy message into PDU format
        memcpy(&msg, socket_buf, sizeof(socket_buf));

        // Handle request for list of available content
        if (msg.type == ONLINE) {
            // Get and parse message from peer
            printf("Online content request\n");
            memcpy(data_recieved, msg.data, sizeof(socket_buf) - 1);
            // Prepare message to send to peer
            send_msg.type = ONLINE;
            for (i=0; i<10; ++i) { //Get list of existing content, seperate by commas
                if (content_tracking[i]==1){
                    strcat(send_msg.data, registered_content[i].content);
                    strcat(send_msg.data, ", ");
                }
            }
            // Send message to peer
            sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin, sizeof(fsin));
        }
        // Handle request to deregister content
        if (msg.type == DEREG) {
            // Get and parse message from peer
            printf("Deregister content request\n");
            memcpy(data_recieved, msg.data, BUFSIZE - 1);
            sscanf(data_recieved, "%[a-zA-Z0-9];%[a-zA-Z0-9]", peer_name,
content_name);

            // Print the peer and content name they want to deregister
            printf("peer name: %s\n", peer_name);
            printf("content name: %s\n", content_name);

            //Check and see if the content exists, and where it's stored
            for (i=0; i<10; ++i){
                if ((strcmp(registered_content[i].content, content_name)==0) &&
(content_tracking[i]==1) && (strcmp(registered_content[i].peer, peer_name)==0)){
                    placement = i;
                    found = 1;
                }
            }
            // If content doesn't exist, send back an error
            if (found == 0){

```

```

        send_msg.type = ERROR;
        strcpy(send_msg.data, "This content is not registered with this peer");
    }
    // If content exists, remove it from storage, and send back an acknowledgement
    else if (found == 1){
        //memset(&registered_content[placement], '\0', sizeof(msg));
        content_tracking[placement] = 0;
        send_msg.type = ACKNOWLEDGEMENT;
        strcpy(send_msg.data, "content has been deregistered");
    }
    // Send message to peer
    sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin, sizeof(fsin));
}
// Handle request to search for content
else if (msg.type == SEARCH){
    // Get and parse message from peer
    printf("Search content request\n");
    memcpy(data_recieved, msg.data, BUFSIZE - 1);
    sscanf(data_recieved, "%[a-zA-Z0-9];%[a-zA-Z0-9]", peer_name,
content_name);

    // Print the peer and content name they want to find
    printf("peer name: %s\n", peer_name);
    printf("content name: %s\n", content_name);

    // Check and see if the content name exists in storage
    for (i=0; i<10; ++i){
        if ((strcmp(registered_content[i].content, content_name)==0) &&
(content_tracking[i]==1)){
            placement = i;
            found = 1;
        }
    }
    // If the content doesn't exist, send back an error
    if (found == 0){
        send_msg.type = ERROR;
        strcpy(send_msg.data, "No content by this name and peer was found");
    }
    //If the content exists, send back an 'S' type message with the address and port
number
    else if (found == 1){
        send_msg.type = SEARCH;
        strcpy(send_msg.data, registered_content[placement].content);
        strcat(send_msg.data, ";");
        strcat(send_msg.data, registered_content[placement].peer);
        strcat(send_msg.data, ";");
        strcat(send_msg.data, registered_content[placement].address);
        strcat(send_msg.data, ";");
        strcat(send_msg.data, registered_content[placement].port_number);
    }
    // Send message to peer

```

```

        sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin, sizeof(fsin));
    }
    // Handle request to quit the program
    else if (msg.type == QUIT){
        // Get and parse the message
        printf("The peer wants to quit the program\n");
        memcpy(peer_name, msg.data, BUFSIZE - 1);
        //Check to see if they have any open content
        for (i=0; i<10; ++i){
            if ((strcmp(registered_content[i].peer, peer_name)==0) &&
content_tracking[i]==1){
                found = 1;
            }
        }
        // If they still have open content, send back an error
        if (found == 1){
            send_msg.type = ERROR;
            strcpy(send_msg.data, "unable to close, content still registered to this
peer");
            sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin,
sizeof(fsin));
        }
        // If all their content is deregistered, send back 'Q' type message
        else if (found == 0){
            send_msg.type = QUIT;
            sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin,
sizeof(fsin));
        }
    }
    // Handle request to register content
    else if (msg.type == REGISTER){
        // Get and parse message
        printf("The peer wants to register new data\n");
        memcpy(data_recieved, msg.data, BUFSIZE - 1);
        sscanf(data_recieved, "%[a-zA-Z0-9];%[a-zA-Z0-9];%[0-9];%[0-9]",
peer_name, content_name, address, port_num);
        //Print information recieved
        printf("peer name: %s\n", peer_name);
        printf("content name: %s\n", content_name);
        printf("address: %s\n", address);
        printf("port: %s\n", port_num);
        //Find an empty spot to store the content information
        for (i=0; i<10; ++i){
            if ((strcmp(registered_content[i].peer, peer_name)==0) &&
(strcmp(registered_content[i].content, content_name)==0) && content_tracking[i]==1){
                error_found = 1;
            }
        }
        if (error_found == 0){
            for (i=0; i<10; ++i){

```

```

        if ((content_tracking[i]==0) && (registered ==0)){
            strcpy(registered_content[i].content, content_name);
            strcpy(registered_content[i].peer, peer_name);
            strcpy(registered_content[i].address, address);
            strcpy(registered_content[i].port_number, port_num);
            registered = 1;
            content_tracking[i] = 1;
        }
    }
}
//Send an acknowledgement if the content was stored properly
if (registered == 1){
    send_msg.type = ACKNOWLEDGEMENT;
    strcpy(send_msg.data, "content has been registered");
}
//Send an error if the content couldn't be stored
else if (registered == 0){
    send_msg.type = ERROR;
    strcpy(send_msg.data, "error registering content");
}
//Send message to the peer
sendto(s, &send_msg, BUFSIZE, 0, (struct sockaddr *)&fsin, sizeof(fsin));
}
}
// Exit the program
close(s);
exit(0);
}

```